1. Introduction

1.1 Company Profile

TrendMart is an in-house project developed as an e-commerce platform similar to Ajio and Myntra. It is designed to provide a user-friendly online shopping experience, offering a variety of products in categories like clothing, accessories, and footwear. This project is developed independently, following professional standards and modern technologies including ReactJS, Node.js, and SQL database management.

1.2 Customer Profile

1.2.1 Current System

Currently, online shoppers rely on manual browsing or basic online marketplaces that often lack personalized features, easy filtering, or order management tools.

1.2.2 Customer Details

The primary users are online shoppers seeking a seamless, interactive, and secure shopping experience. TrendMart allows dynamic product display, secure payments, order tracking, and customer account management. The system simulates a real-world e-commerce platform and is designed to demonstrate a fully functional online retail solution.

2. Proposed System

2.1 Scope

The TrendMart system is designed to provide a complete e-commerce solution for online shoppers. It handles user registration, product browsing, filtering, cart management, order placement, payment processing, and review submission. The system supports multiple product categories, variants, and sellers. By automating the shopping workflow, it reduces manual effort and provides a seamless, interactive online shopping experience using .NET, ReactJS, and SQL Server.

2.2 Objective

- To develop a user-friendly e-commerce platform similar to Ajio and Myntra.

- To allow dynamic product management with categories, variants, and images.

- To enable users to add products to cart, place orders, and make payments securely.

- To provide order tracking and history for users.

- To allow users to review products and maintain wishlists.

- To manage user profiles and addresses efficiently.

2.3 Constraints

2.3.1 H/w Constraints

- Requires a system with at least 4 GB RAM and 2 GHz processor.

- Minimum 500 MB free disk space for application deployment.

2.3.2 S/W Constraints

- The system requires .NET (backend), ReactJS (frontend), SQL Server (database), and modern web browsers.

- Mobile responsiveness may vary depending on device compatibility.

2.4 Advantages

- Automated order and payment processing reduces manual errors.

- Supports multiple users and sellers with secure account management.

- Dynamic product management allows real-time updates.

- Interactive and user-friendly interface enhances customer satisfaction.

2.5 Limitations

- Currently limited to web platform only; mobile apps are not included.

- Advanced analytics and AI-based recommendations are not implemented.

- Some third-party integrations like payment gateways may require future enhancements.

3. Environment Specification

3.1 Hardware & Software Requirements

- Hardware Requirements:

  - Processor: Minimum 2 GHz dual-core or higher

  - RAM: 4 GB or more

  - Hard Disk: 500 MB free space minimum

  - Display: 1366x768 resolution or higher

- Software Requirements:

  - Backend: .NET framework / .NET Core

  - Frontend: ReactJS, Node.js (for development tooling)

  - Database: SQL Server 2019 or later

  - Web Browsers: Google Chrome, Mozilla Firefox, Microsoft Edge (latest versions)

  - IDE / Tools: Visual Studio 2022 / VS Code, SQL Server Management Studio (SSMS)

3.2 Development Description

The TrendMart system is developed as a web-based e-commerce platform. The backend is implemented using .NET, providing APIs for user management, product management, orders, and payments. The frontend is developed using ReactJS, ensuring a responsive and interactive user interface. The SQL Server database is used to store and manage all data, including users, products, orders, payments, and reviews. The system follows a modular approach, separating different functionalities into components for maintainability and scalability.

4. System Planning

4.1 Feasibility Study

The TrendMart project is technically and operationally feasible. The system leverages .NET, ReactJS, and SQL Server, which are widely supported technologies. The project can be implemented within the in-house resources and timeline, and it meets the operational requirements of an e-commerce platform, including user management, product handling, orders, and payments.

4.2 Software Engineering Model

The project follows the Agile development model, allowing iterative development and continuous testing. Each module (User Management, Product Management, Orders, Payments) is developed as a separate component and integrated progressively. Agile ensures flexibility for changes and supports incremental delivery.

4.3 Risk Analysis

- Technical Risks: Integration issues between frontend (ReactJS) and backend (.NET)

- Operational Risks: Delays in testing or deployment

- Data Risks: Database corruption or unauthorized access

- Mitigation: Proper version control, regular backups, and secure authentication mechanisms are implemented

4.4 Project Schedule

4.4.1 Task Dependency

- User Management → Product Management → Orders → Payments → Reviews

- Frontend design → Backend APIs → Database Integration → Testing

4.4.2 Timeline Chart

- Week 1–2: Requirement Analysis & Environment Setup

- Week 3–4: Database Design & Backend API Development

- Week 5–6: Frontend Development (ReactJS)

- Week 7: Integration & Testing

- Week 8: Deployment & Documentation

4.4.3 Project Table

Task	Duration	Start Week	End Week	Dependencies
Requirement Analysis	2 weeks	1	2	None
Database & Backend	2 weeks	3	4	Requirement Analysis
Frontend Development	2 weeks	5	6	Database & Backend
Integration & Testing	1 week	7	7	Frontend Development
Deployment & Documentation	1 week	8	8	Integration & Testing

5. System Analysis

5.1 Detailed SRS (Module-wise specification)

Module: User Management

- Description: Handles user registration, login, profile management, and account settings.

- Input(s): User details like name, email, password, contact number, profile info.

- Events: User registration, login attempts, profile updates.

- Output(s): Confirmation messages, user account data, login status.

- Validations: Email format, password strength, mandatory fields, unique email.

- Constraints: Maximum profile picture size 2MB, supported image formats: jpg/png.

Module: Product Management

- Description: Manages product catalog, categories, product variants, and images.

- Input(s): Product name, description, price, category, images, variant details.

- Events: Add product, update product, delete product, upload images.

- Output(s): Confirmation messages, updated product listings.

- Validations: Price numeric, required fields check, image format validation.

- Constraints: Maximum 10 images per product, stock quantity non-negative.

Module: Order Management

- Description: Handles order placement, cart management, and order tracking.
- Input(s): Selected products, quantities, user address, payment method.
- Events: Add to cart, place order, update order status.
- Output(s): Order confirmation, payment receipt, order history.
- Validations: Stock availability, valid payment information.
- Constraints: Order cancellation allowed only before shipment.

Module: Payment Management

- Description: Processes payments for orders.
- Input(s): Payment method, order ID, transaction details.
- Events: Payment initiation, verification, confirmation.
- Output(s): Payment success/failure, receipt generation.
- Validations: Payment amount, valid method, transaction integrity.
- Constraints: Only online payments supported; refunds handled manually.

Module: Reviews & Wishlist

- Description: Allows users to add reviews, ratings, and maintain wishlist.

- Input(s): Product ID, rating, comments.

- Events: Add review, update review, add/remove from wishlist.

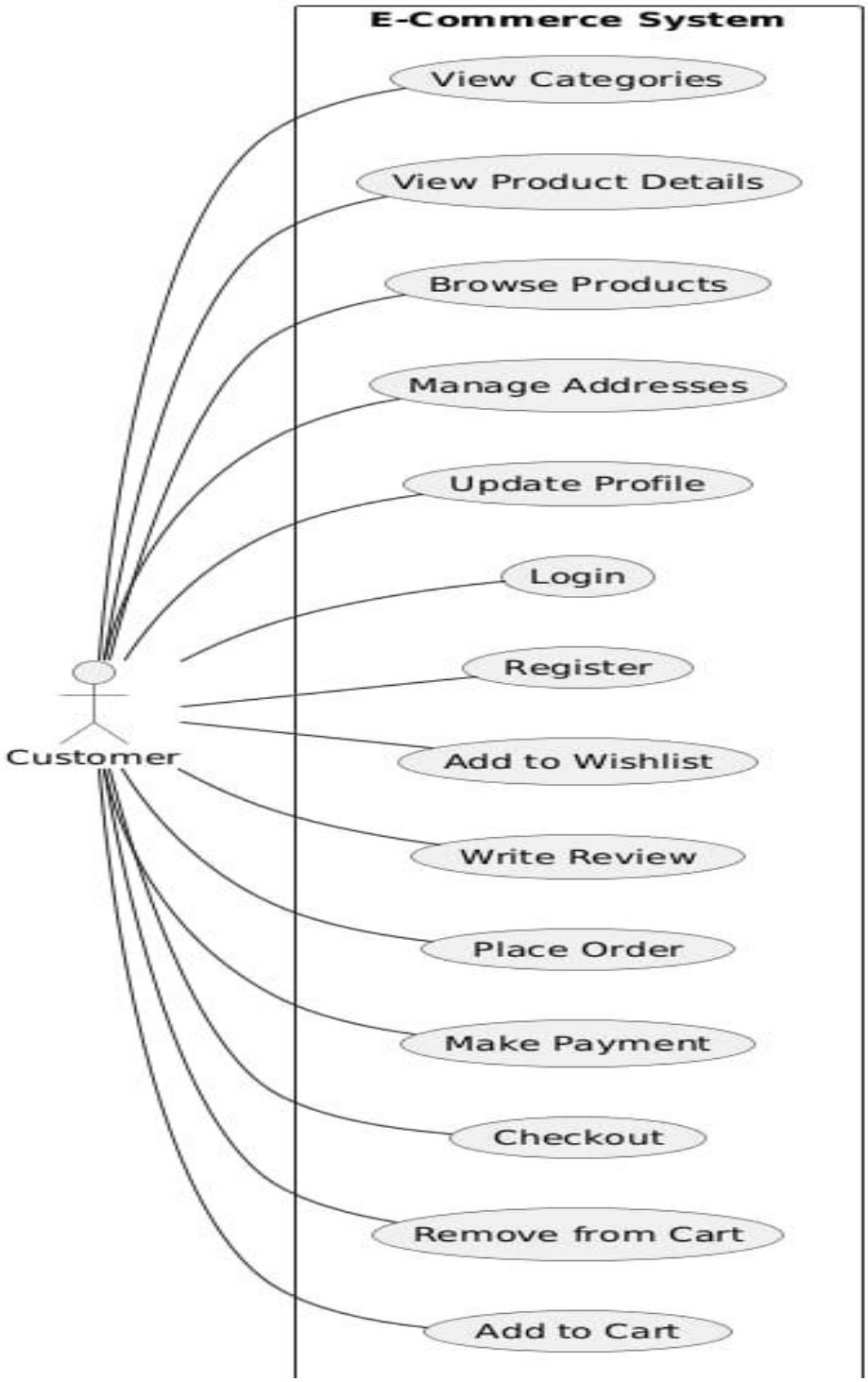
- Output(s): Confirmation messages, updated wishlist, displayed reviews.

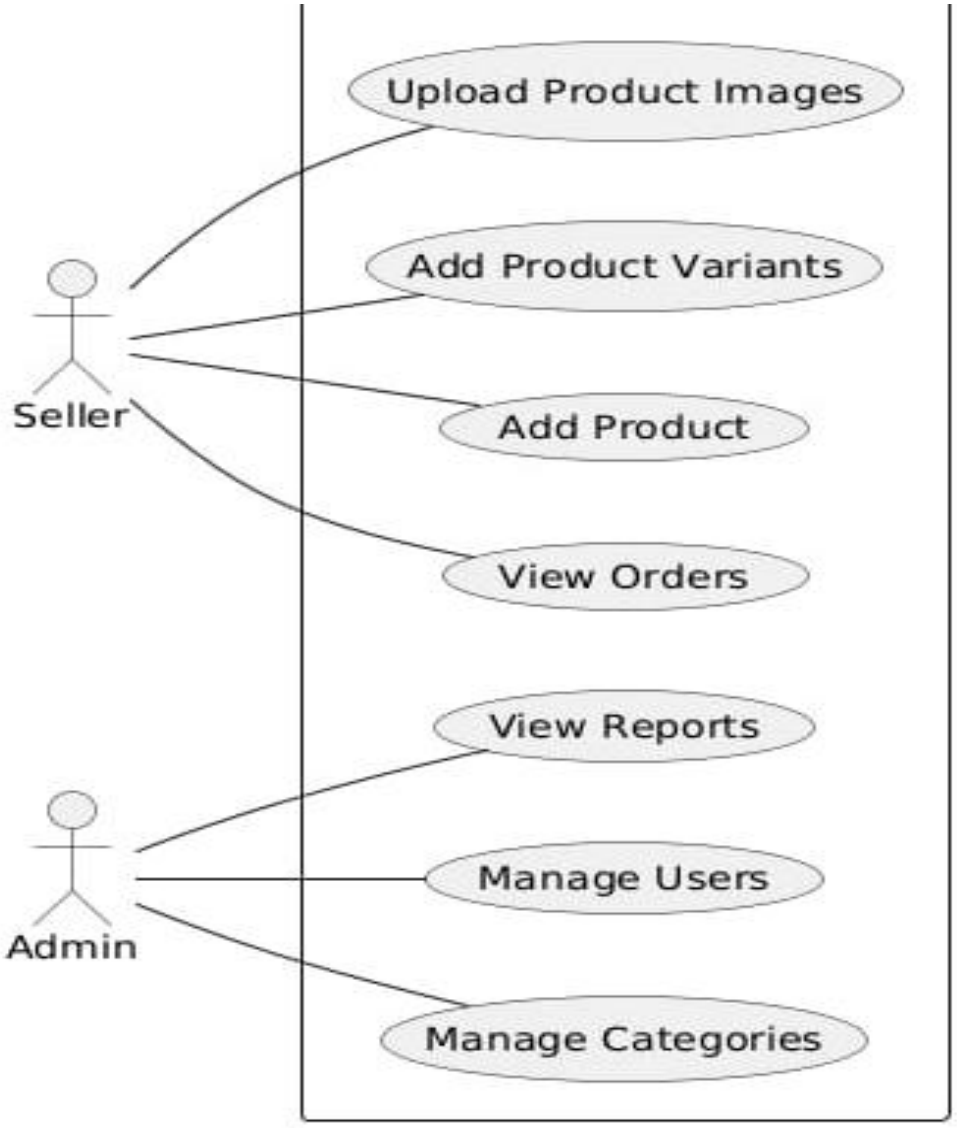
- Validations: Rating between 1–5, non-empty comments.

- Constraints: One review per product per user.

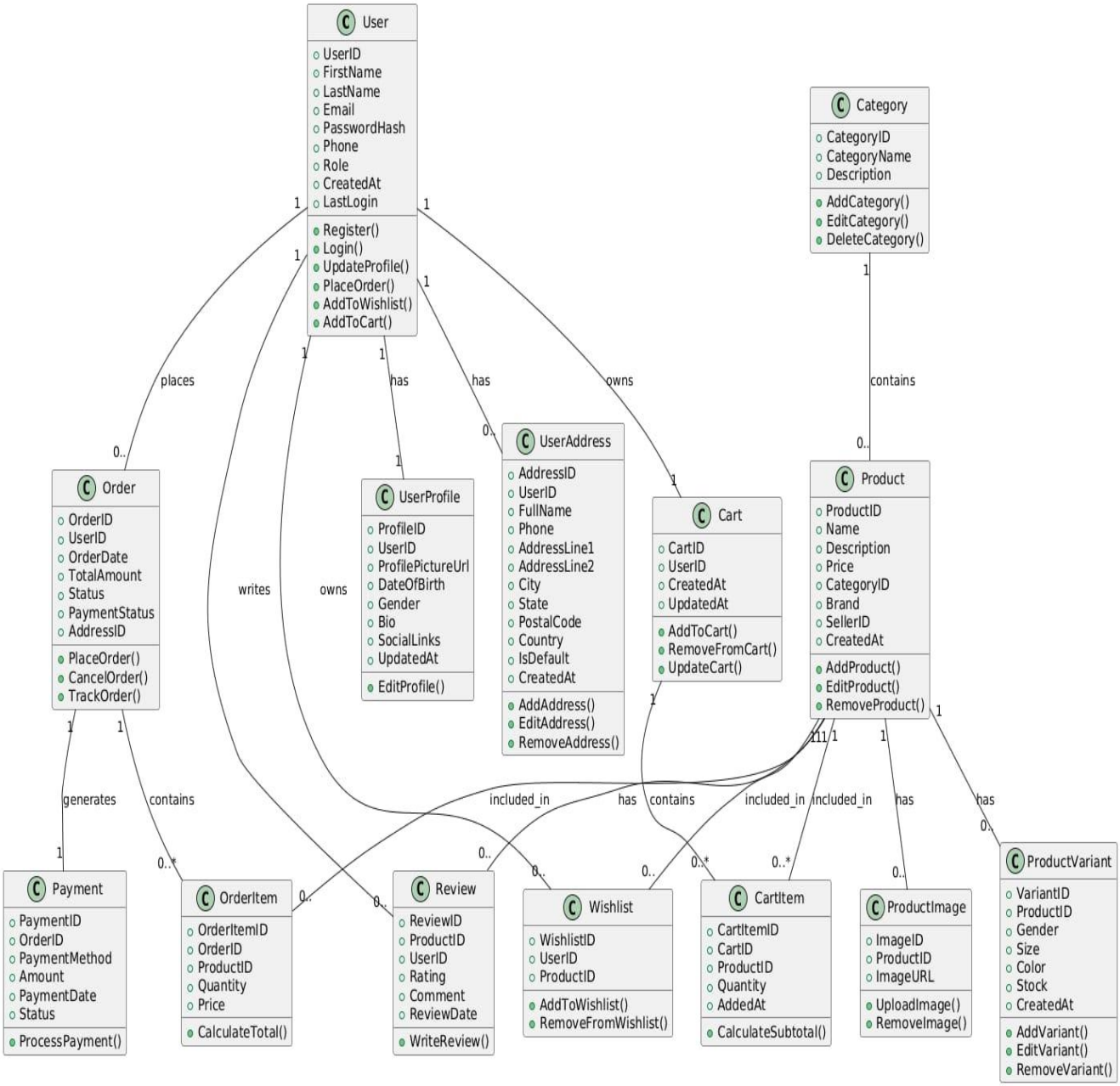
5.2 UML Diagrams

5.2.1 Use Case Diagram:

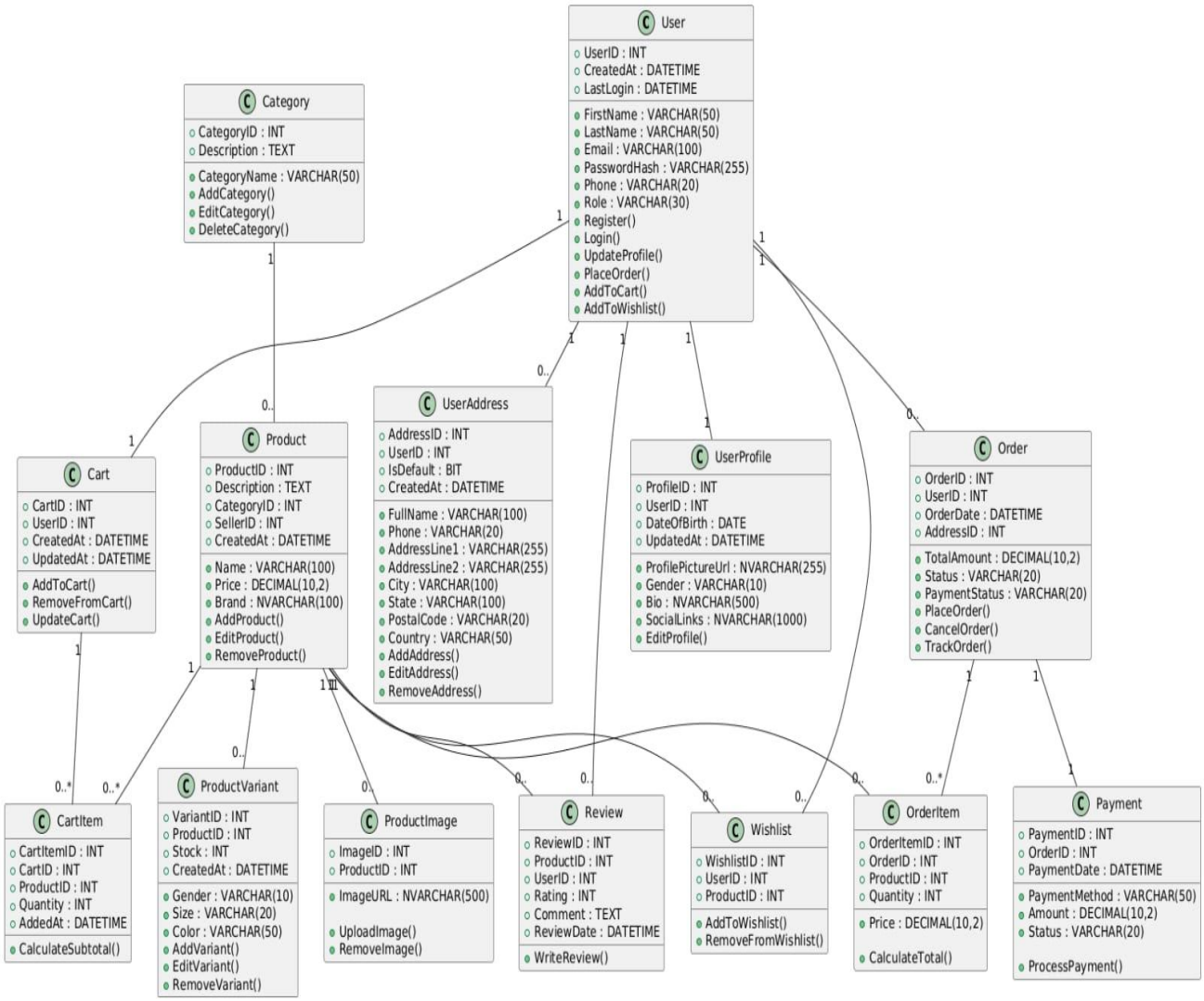



Upload Product Images
Add Product Variants
Add Product
View Orders
View Reports
Manage Users
Manage Categories
Seller
Admin

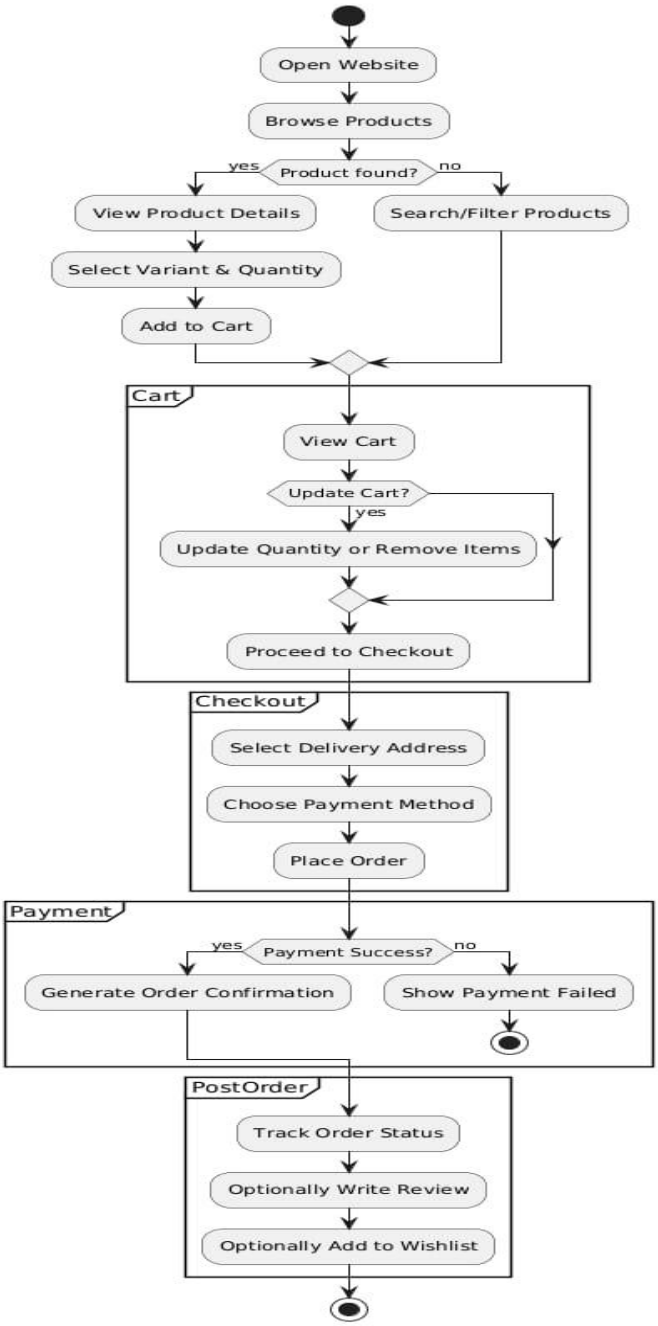
5.2.2 CRC (Class-Responsibility-Collaborator):



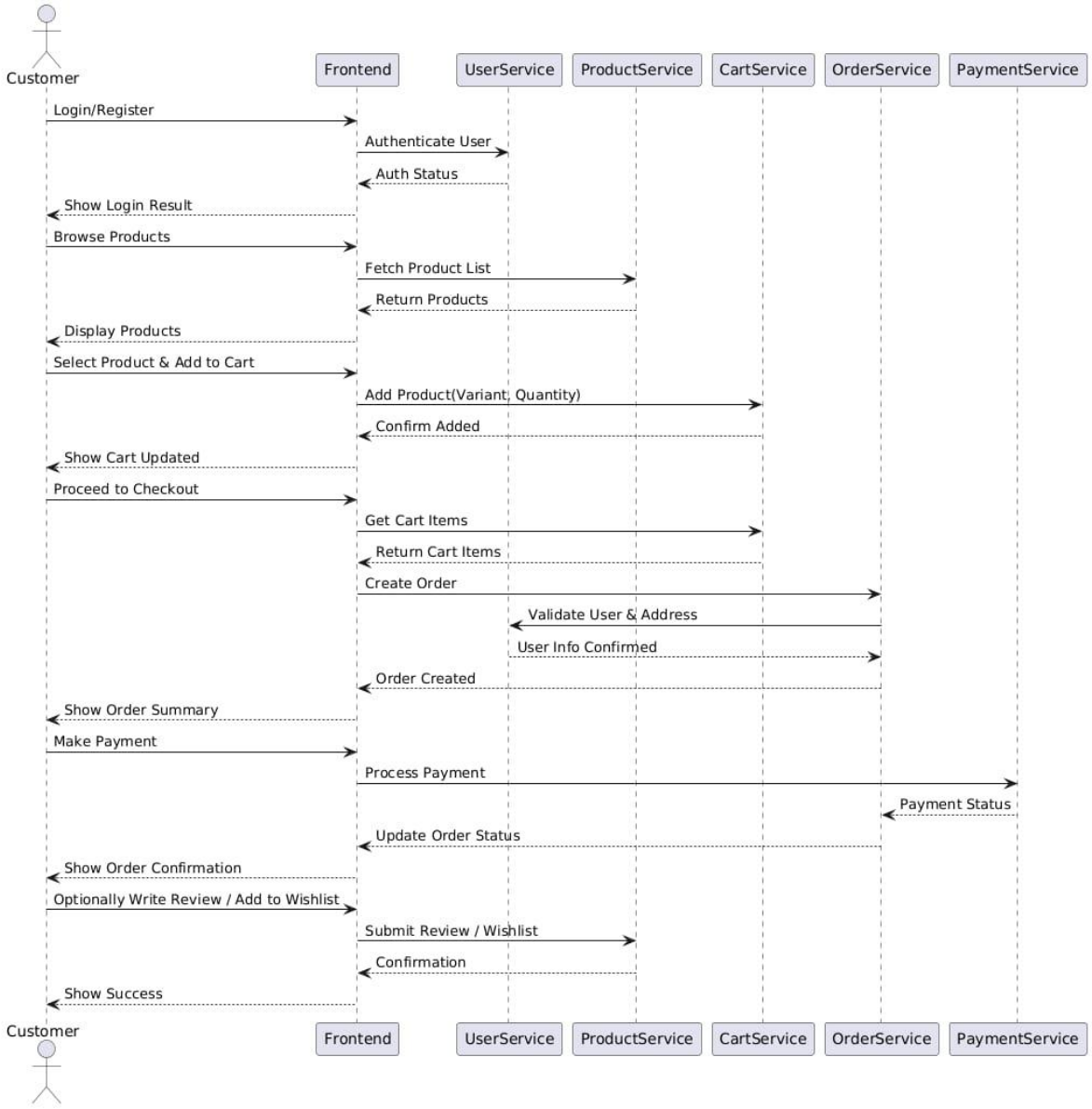
5.2.3 Class Diagram: Placeholder:



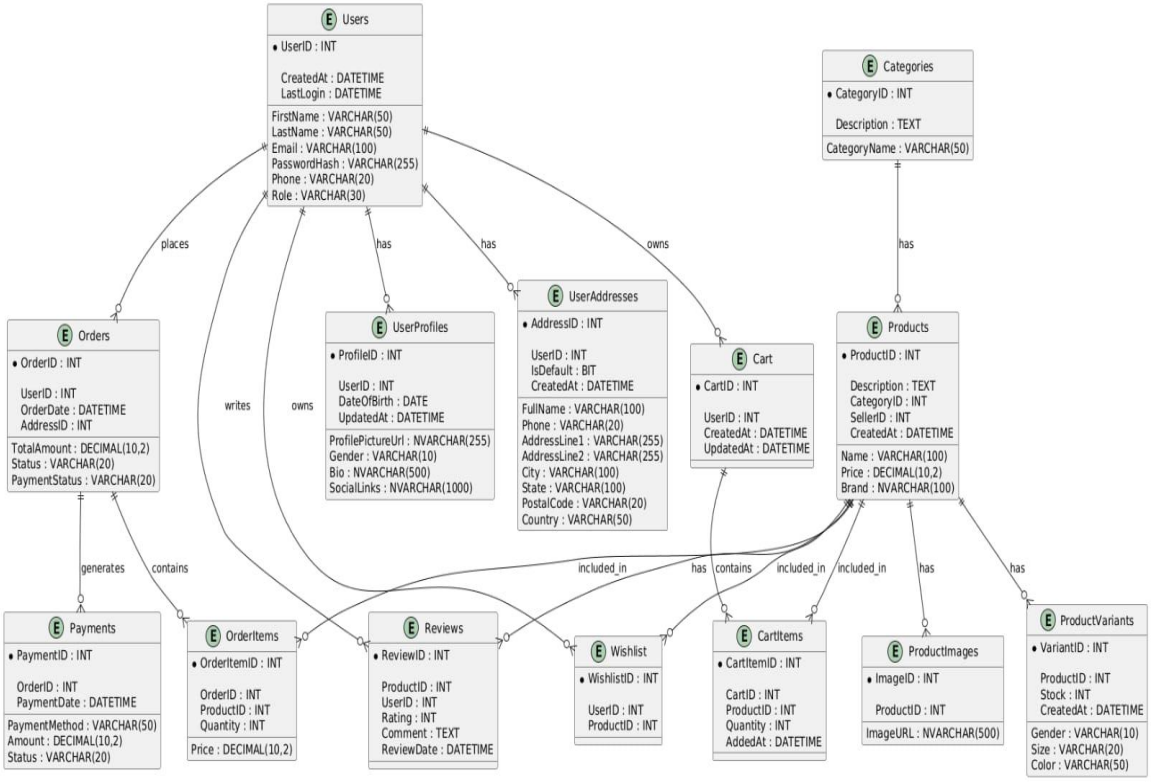
5.2.4 Activity Diagram: Placeholder:



5.2.5 Sequence Diagram: Placeholder:



5.3 E-R Diagram



6. Software Design

6.1 Database Design

Users Table

Column Name	Type	Key
UserID	INT	Primary Key
FirstName	VARCHAR(50)	-
LastName	VARCHAR(50)	-
Email	VARCHAR(100)	Unique
PasswordHash	VARCHAR(255)	-
Phone	VARCHAR(20)	-
Role	VARCHAR(30)	-
CreatedAt	DATETIME	-
LastLogin	DATETIME	-

UserProfiles

Column Name	Type	Key
ProfileID	INT	Primary Key
UserID	INT	Foreign Key
ProfilePictureUrl	NVARCHAR(255)	-
DateOfBirth	DATE	-
Gender	VARCHAR(10)	-
Bio	NVARCHAR(500)	-
SocialLinks	NVARCHAR(1000)	-
UpdatedAt	DATETIME	-

UserAddresses Table

Column Name	Type	Key
AddressID	INT	Primary Key
UserID	INT	Foreign Key
FullName	VARCHAR(100)	-
Phone	VARCHAR(20)	-
AddressLine1	VARCHAR(255)	-
AddressLine2	VARCHAR(255)	-
City	VARCHAR(100)	-
State	VARCHAR(100)	-
PostalCode	VARCHAR(20)	-
Country	VARCHAR(50)	-
IsDefault	BIT	-
CreatedAt	DATETIME	-

Categories Table

Column Name	Type	Key
CategoryID	INT	Primary Key
CategoryName	VARCHAR(50)	Unique
Description	TEXT	-

Products Table

Column Name	Type	Key
ProductID	INT	Primary Key
Name	VARCHAR(100)	-
Description	TEXT	-
Price	DECIMAL(10,2)	-
CategoryID	INT	Foreign Key
Brand	NVARCHAR(100)	-
SellerID	INT	Foreign Key
CreatedAt	DATETIME	-

ProductImages Table

Column Name	Type	Key
ImageID	INT	Primary Key
ProductID	INT	Foreign Key
ImageURL	NVARCHAR(500)	-

ProductVariants Table

Column Name	Type	Key
VariantID	INT	Primary Key
ProductID	INT	Foreign Key
Gender	VARCHAR(10)	-
Size	VARCHAR(20)	-
Color	VARCHAR(50)	-
Stock	INT	-
CreatedAt	DATETIME	-

Orders Table

Column Name	Type	Key
OrderID	INT	Primary Key
UserID	INT	Foreign Key
OrderDate	DATETIME	-
TotalAmount	DECIMAL(10,2)	-
Status	VARCHAR(20)	-
PaymentStatus	VARCHAR(20)	-
AddressID	INT	Foreign Key

OrderItems Table

Column Name	Type	Key
OrderItemID	INT	Primary Key
OrderID	INT	Foreign Key
VariantID	INT	Foreign Key
Quantity	INT	-
Price	DECIMAL(10,2)	-

Payments Table

Column Name	Type	Key
PaymentID	INT	Primary Key
OrderID	INT	Foreign Key
PaymentMethod	VARCHAR(50)	-
Amount	DECIMAL(10,2)	-
PaymentDate	DATETIME	-
Status	VARCHAR(20)	-

Reviews Table

Column Name	Type	Key
ReviewID	INT	Primary Key
ProductID	INT	Foreign Key
UserID	INT	Foreign Key
Rating	INT	-
Comment	TEXT	-
ReviewDate	DATETIME	-

Wishlist Table

Column Name	Type	Key
WishlistID	INT	Primary Key
UserID	INT	Foreign Key
ProductID	INT	Foreign Key

Cart Table

Column Name	Type	Key
CartID	INT	Primary Key
UserID	INT	Foreign Key
CreatedAt	DATETIME	-
UpdatedAt	DATETIME	-

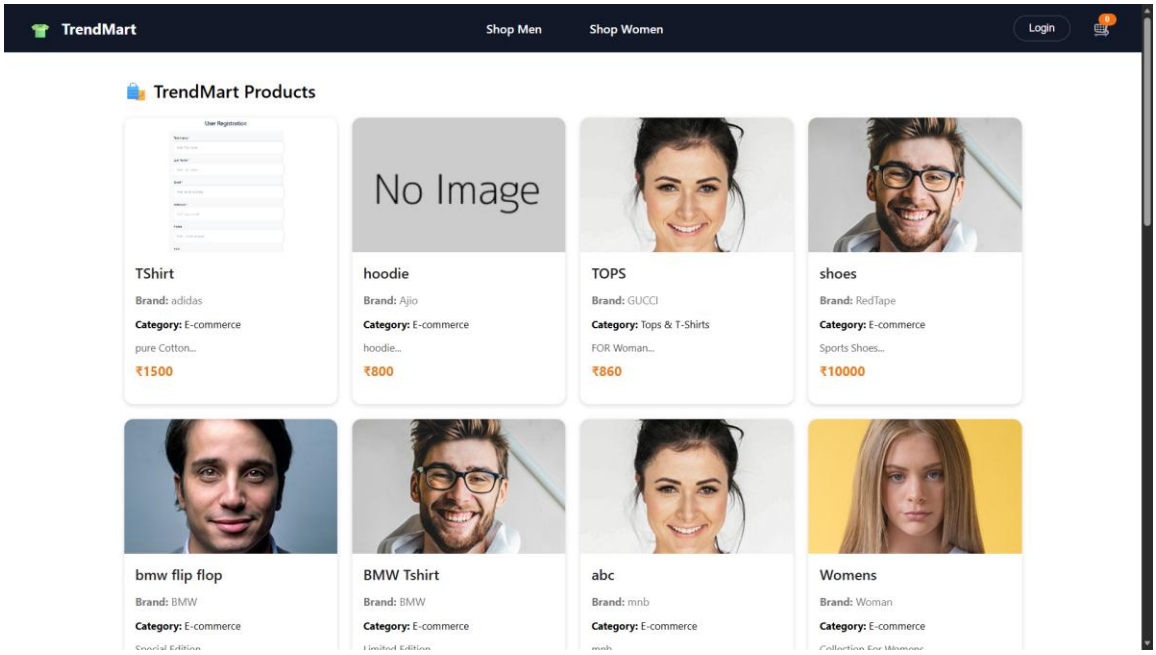
CartItems Table

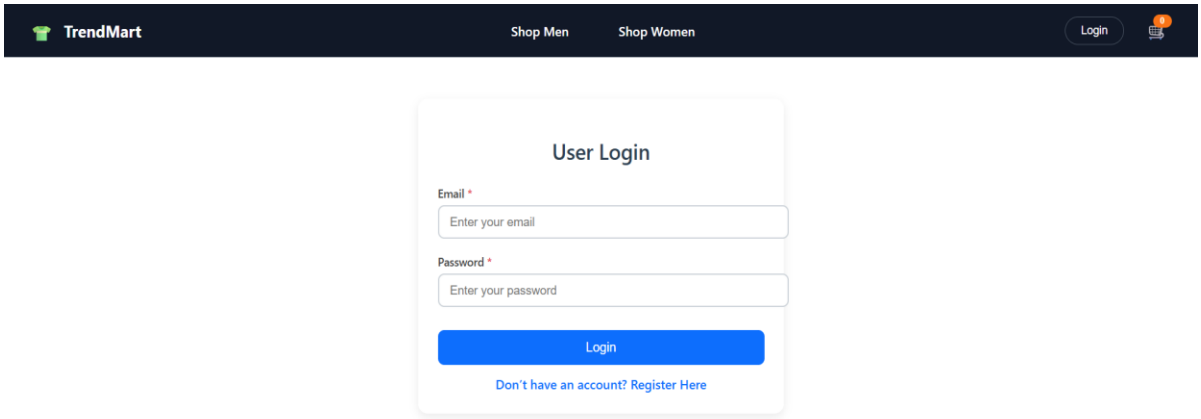
Column Name	Type	Key
CartItemID	INT	Primary Key
CartID	INT	Foreign Key
VariantID	INT	Foreign Key
Quantity	INT	-
AddedAt	DATETIME	-

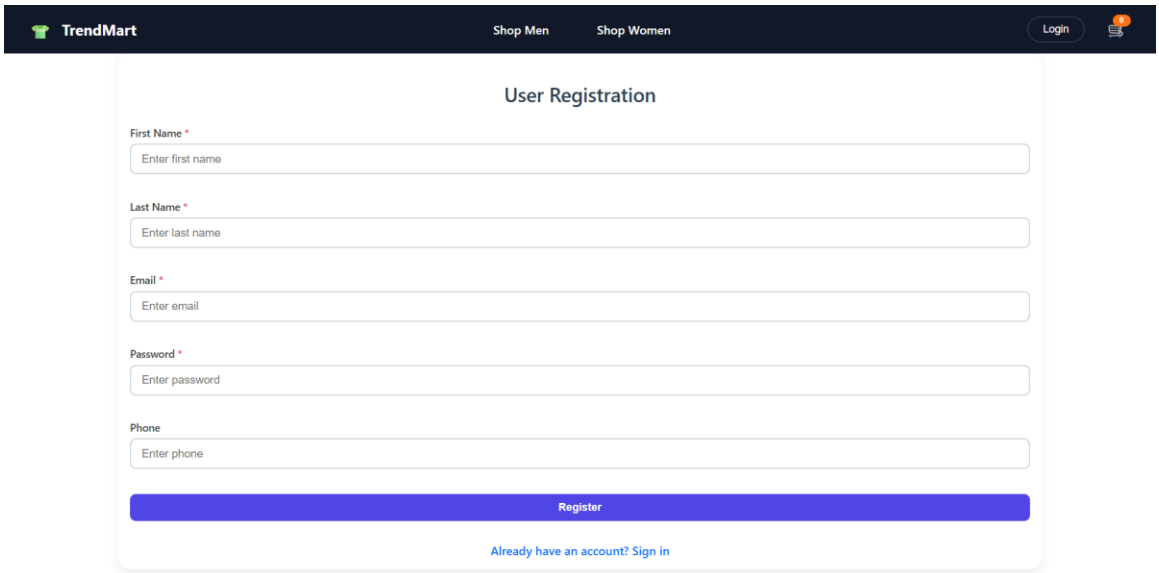
6.2 Interface Design (Sitemap & Page Snapshots)

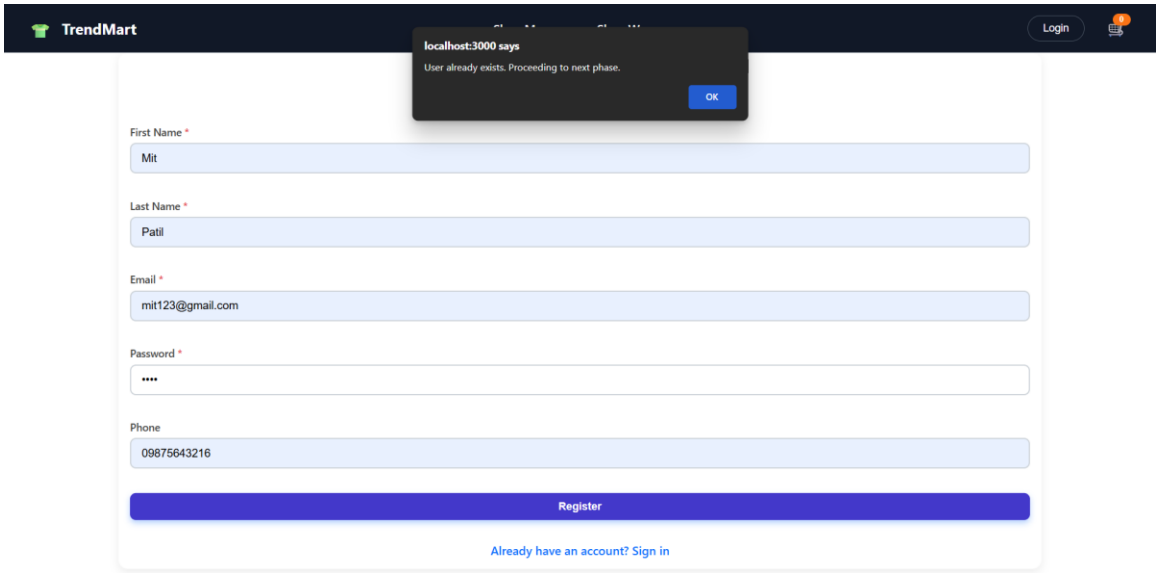
The TrendMart system frontend is developed using **ReactJS**, following a modular and responsive design. Key pages include:

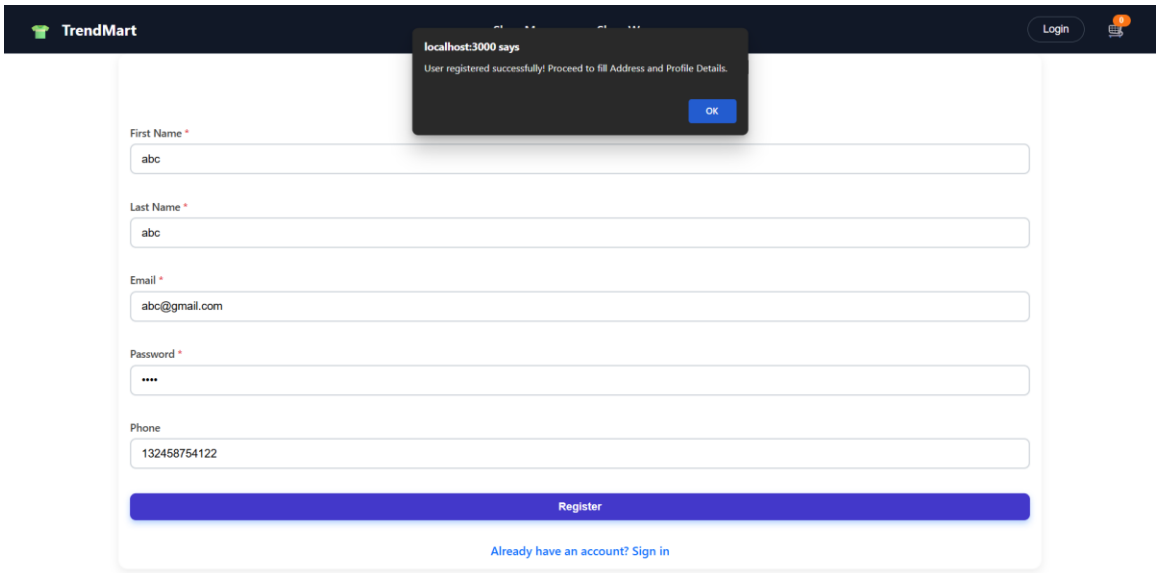
- **Home Page** – Displays product categories, featured products, and promotions.

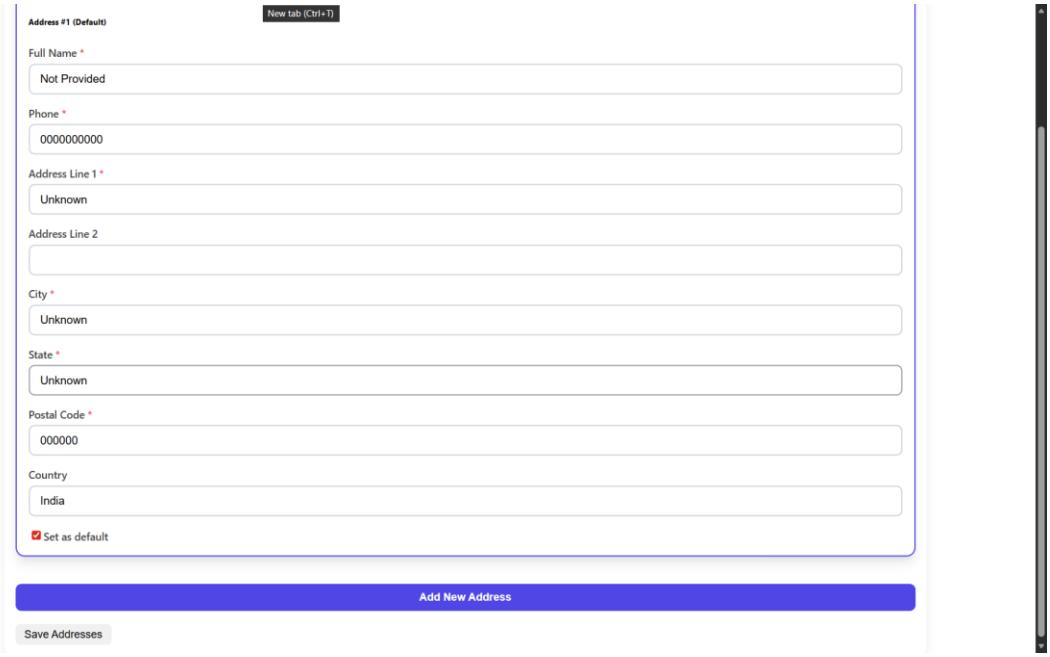


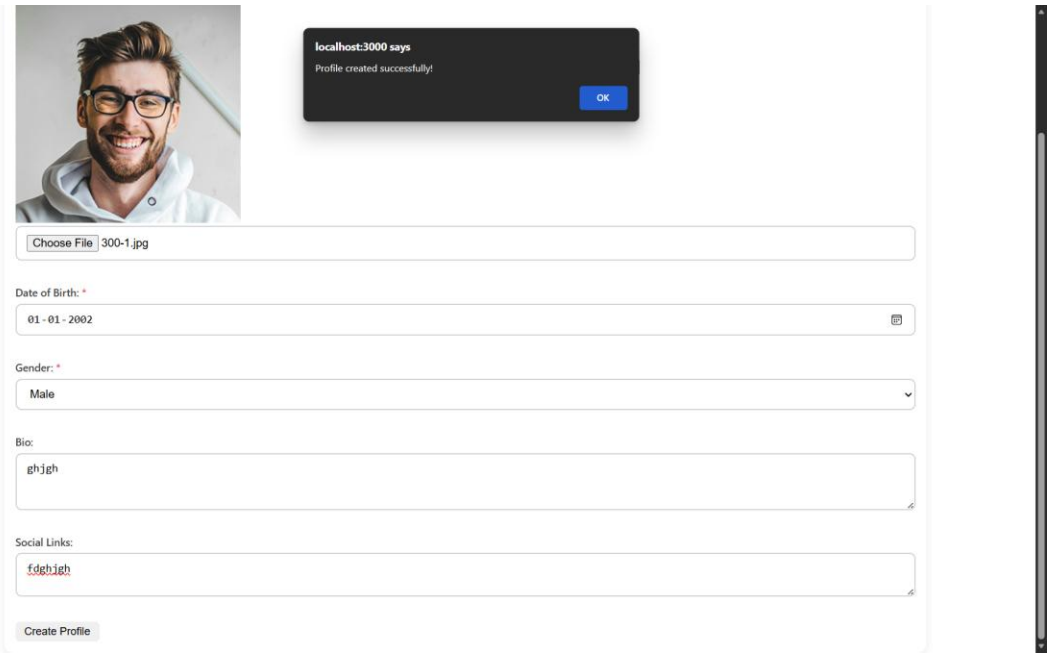

TrendMart
Shop Men
Shop Women
Login
User Login
Email *
Enter your email
Password *
Enter your password
Login
Don't have an account? Register Here

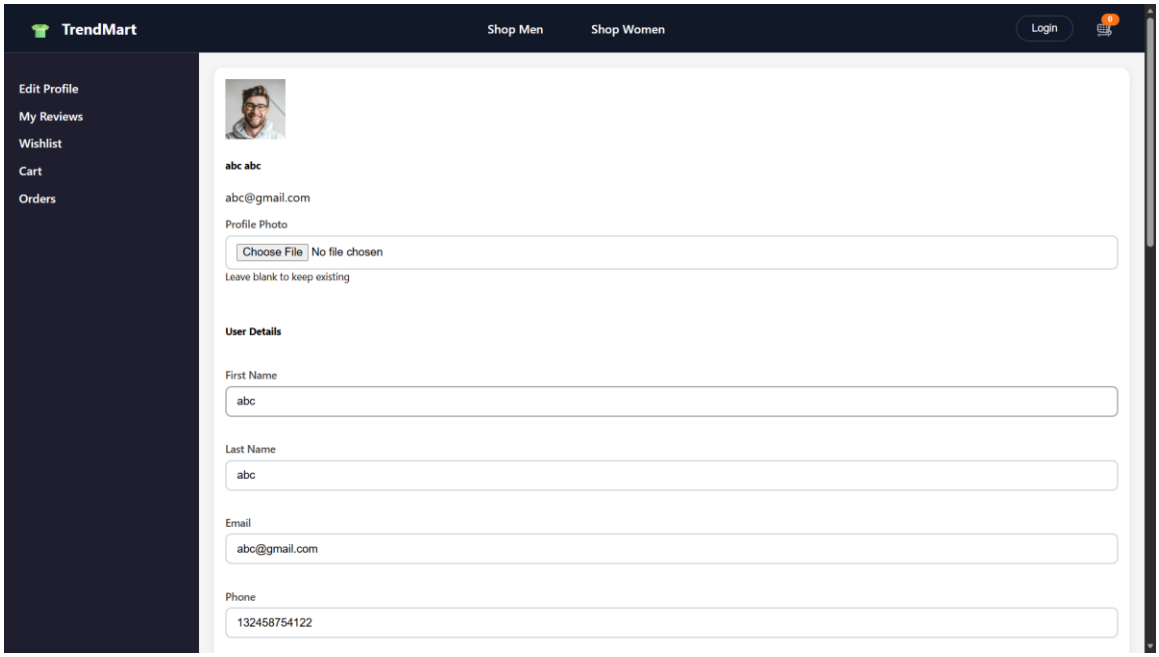

TrendMart
Shop Men
Shop Women
Login
User Registration
First Name *
Enter first name
Last Name *
Enter last name
Email *
Enter email
Password *
Enter password
Phone
Enter phone
Register
Already have an account? Sign in


TrendMart
Login
localhost:3000 says
User already exists. Proceeding to next phase.
OK
First Name *
Mit
Last Name *
Patil
Email *
mit123@gmail.com
Password *
••••
Phone
09875643216
Register
Already have an account? Sign in

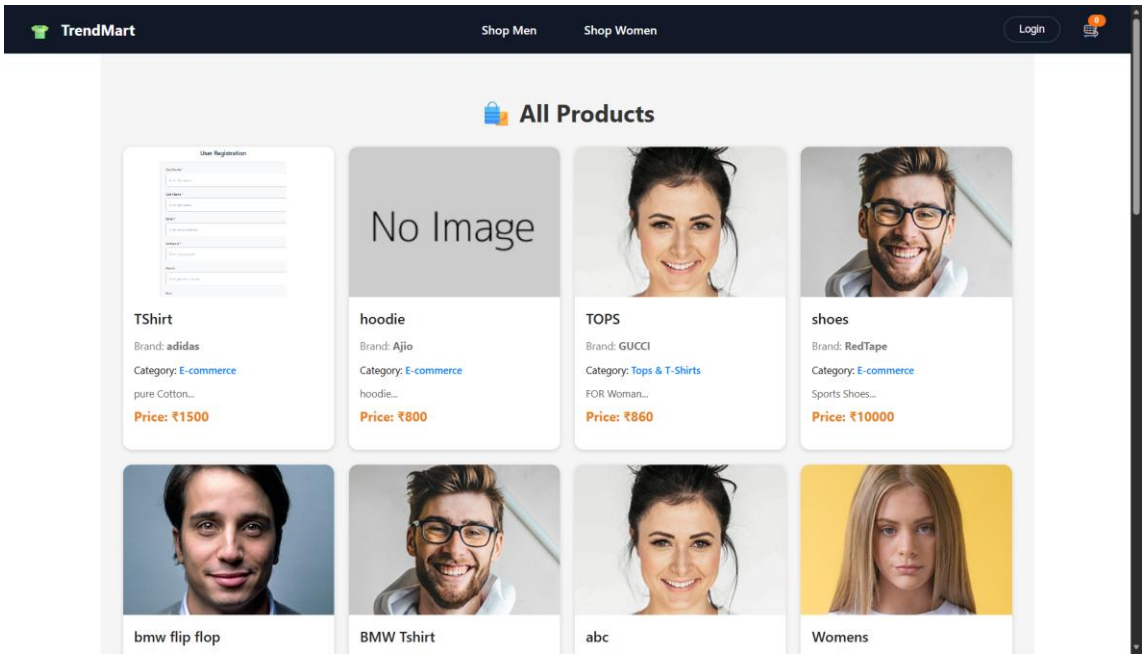

TrendMart
Login
localhost:3000 says
User registered successfully! Proceed to fill Address and Profile Details.
OK
First Name *
abc
Last Name *
abc
Email *
abc@gmail.com
Password *
••••
Phone
132458754122
Register
Already have an account? Sign in

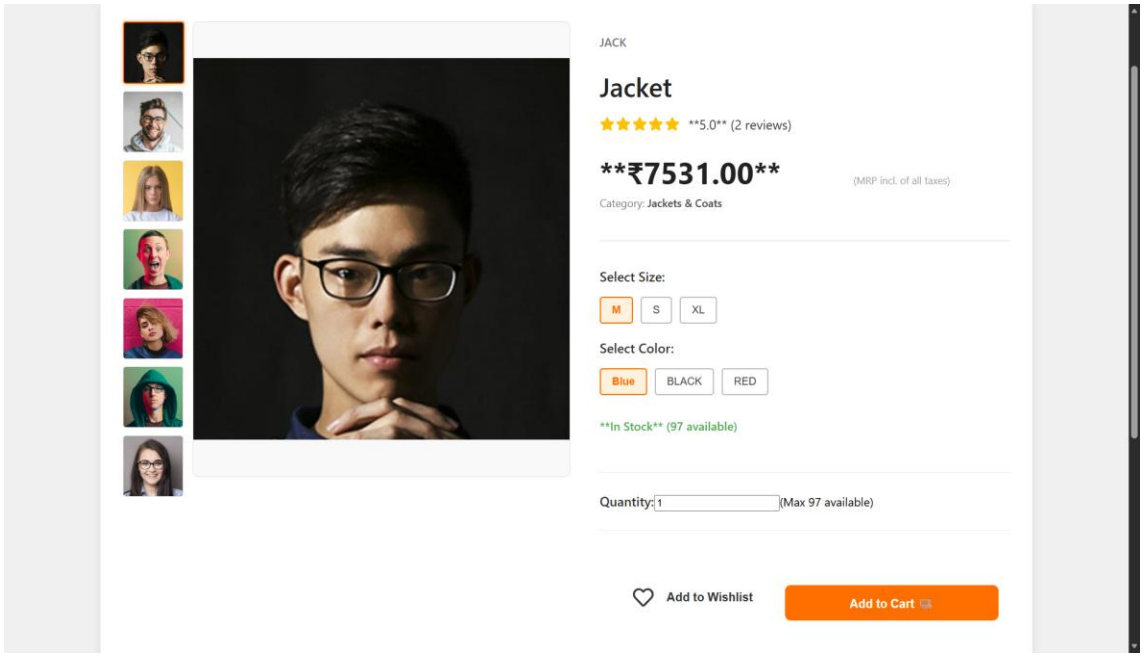

Address #1 (Default)
New tab (Ctrl+T)
Full Name *
Not Provided
Phone *
0000000000
Address Line 1 *
Unknown
Address Line 2
City *
Unknown
State *
Unknown
Postal Code *
000000
Country
India
Set as default
Add New Address
Save Addresses

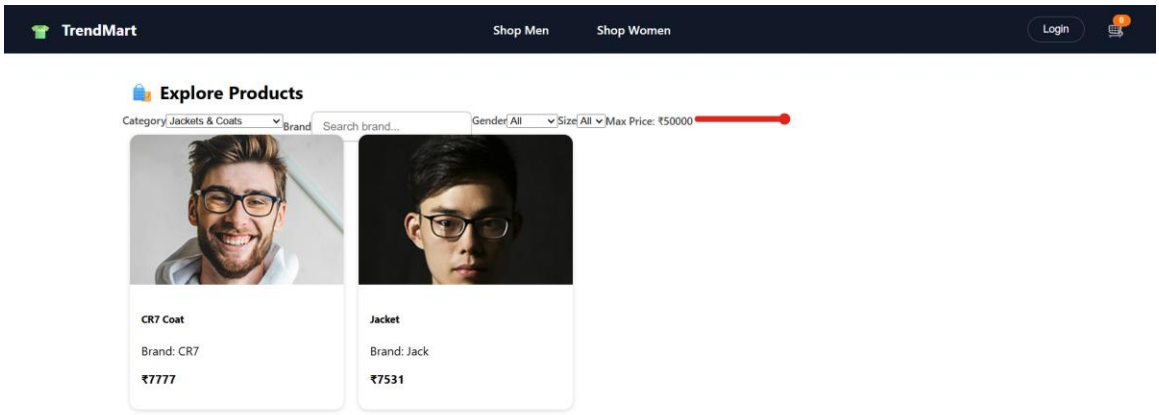

localhost:3000 says
Profile created successfully!
OK
Choose File 300-1.jpg
Date of Birth: *
01-01-2002
Gender: *
Male
Bio:
ghjgh
Social Links:
fdghjgh
Create Profile


TrendMart
Shop Men
Shop Women
Login
Edit Profile
My Reviews
Wishlist
Cart
Orders
abc abc
abc@gmail.com
Profile Photo
Choose File No file chosen
Leave blank to keep existing
User Details
First Name
abc
Last Name
abc
Email
abc@gmail.com
Phone
132458754122

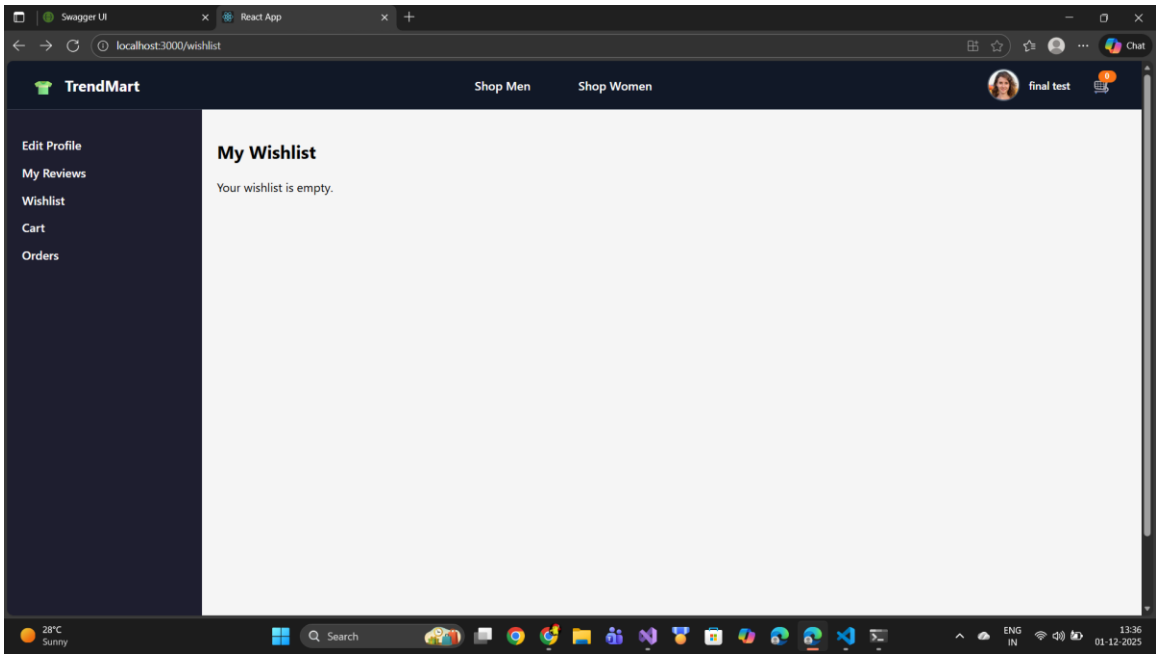
- **Product Listing Page** – Allows filtering by category, brand, size, price, and gender.

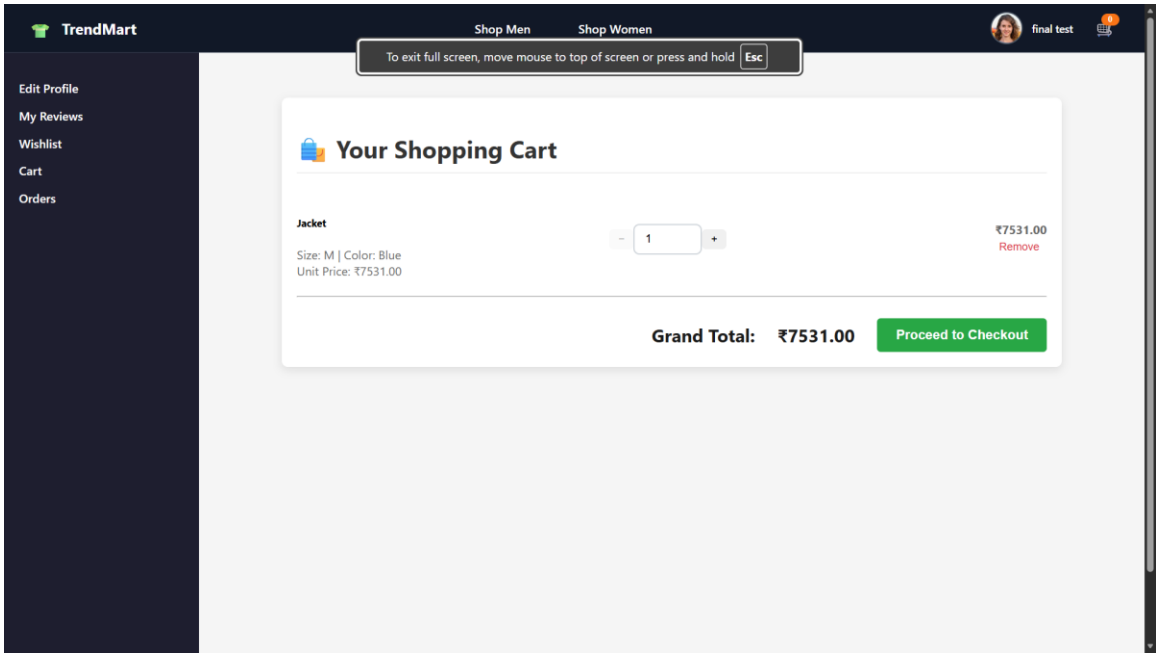


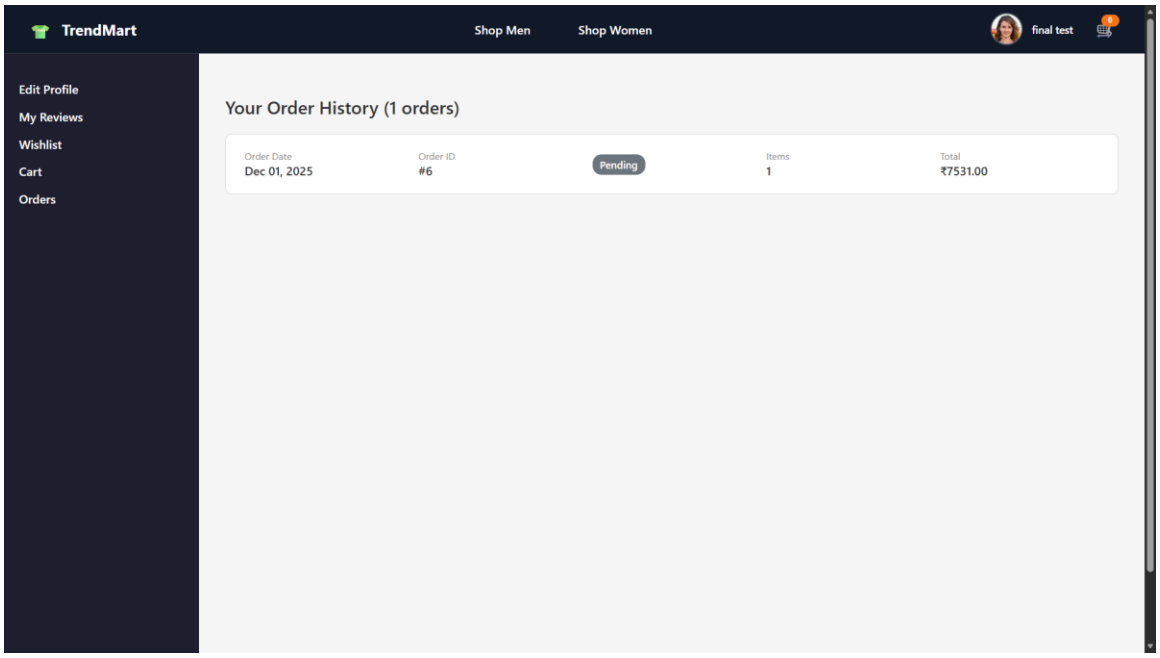
- 


TrendMart
Shop Men
Shop Women
Login
Explore Products
Category Jackets & Coats
Brand Search brand...
Gender All
Size All
Max Price: ₹50000
CR7 Coat
Brand: CR7
₹7777
Jacket
Brand: Jack
₹7531

- **Cart & Checkout Page** – Displays selected products, quantities, and payment options.






TrendMart
Shop Men
Shop Women
final test
Edit Profile
My Reviews
Wishlist
Cart
Orders
Your Order History (1 orders)
Order Date
Dec 01, 2025
Order ID
#6
Pending
Items
1
Total
₹7531.00

6.3 Architecture Design

The TrendMart system follows a **3-tier architecture**:

1. **Presentation Layer (Frontend)** – ReactJS components handle user interface and interaction.
2. **Business Logic Layer (Backend)** – .NET APIs handle data processing, business rules, and request validation.
3. **Data Layer (Database)** – SQL Server stores all persistent data, including users, products, orders, and payments.

This architecture ensures **modularity, maintainability, and scalability**, allowing independent development and deployment of each layer.

7. Testing

7.1 Unit Testing

Unit testing is performed to verify that individual modules of the TrendMart system function correctly. Each component is tested independently to ensure that it meets the specified requirements:

- **User Management Module:** Tests for user registration, login, profile update, and validation of inputs.
- **Product Management Module:** Tests for adding, updating, deleting products, and uploading images and variants.
- **Order & Cart Module:** Tests for adding items to cart, placing orders, calculating totals, and updating quantities.
- **Payment Module:** Tests for processing payments, verifying amounts, and updating payment status.
- **Reviews & Wishlist Module:** Tests for submitting reviews, ratings, and maintaining wishlist items.

Unit tests were performed using **.NET Unit Testing Framework** and **manual frontend checks** in ReactJS.

7.2 Integration Testing

Integration testing ensures that modules interact correctly and data flows seamlessly across the system:

- **Frontend–Backend Integration:** Validates that ReactJS components correctly call .NET APIs and display data.
- **Database Integration:** Confirms that CRUD operations on SQL Server tables work correctly for users, products, orders, and payments.
- **End-to-End Workflows:** Tests complete user actions such as browsing products, adding to cart, placing orders, making payments, and submitting reviews.

Integration testing helps identify **data mismatches, API errors, and communication issues** between different modules before deployment.

8. Future Enhancement

- Implement **mobile applications** for Android and iOS to increase accessibility.
- Add **AI-based product recommendations** for personalized shopping experiences.
- Integrate **multiple payment gateways** for more convenience.
- Introduce **real-time order tracking** and notifications.
- Implement **advanced analytics** for sales trends, inventory, and customer behavior.
- Add **multi-language support** for a wider user base.

9. Glossary

Term	Definition
API	Application Programming Interface, used to communicate between frontend and backend.
CRUD	Create, Read, Update, Delete – basic operations on data in the database.
ReactJS	JavaScript library for building responsive user interfaces.
.NET	Framework used to develop backend APIs and business logic.
SQL Server	Relational database system used for data storage and management.
Variant	Different options of a product, e.g., size, color, or gender.
Agile	Software development methodology focused on iterative progress.
ER Diagram	Entity-Relationship Diagram, used to visualize database structure.

10. Reference

1. Official **ReactJS Documentation** – https://reactjs.org/docs/getting-started.html
2. Official **.NET Documentation** – https://learn.microsoft.com/en-us/dotnet/
3. SQL Server Documentation – https://learn.microsoft.com/en-us/sql/sql-server/
4. W3Schools – https://www.w3schools.com/ (For general web development reference)
5. Stack Overflow – https://stackoverflow.com/ (For problem-solving and code examples)